

March_11 LP

March 11, 2024

```
[1]: from sklearn.datasets import load_iris
      from sklearn.tree import DecisionTreeClassifier
      from sklearn.model_selection import train_test_split
      from sklearn.metrics import accuracy_score
```

```
[2]: #Load the iris dataset
      #Split the dataset into training and testing sets
      #Create a DT classifier
      #Expose the Created DT Classifier into Training dataset
      #Predict the labels of the test dataset
      #Evaluate the performance of the model
```

```
[12]: #Load the iris dataset
      iris = load_iris()
      X = iris.data
      y = iris.target

      #Split the dataset into training and testing sets
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25,
      ↪random_state = 42)

      #Create a DT classifier
      model = DecisionTreeClassifier()

      #Expose the Created DT Classifier into Training dataset
      model.fit(X_train, y_train)

      #Predict the labels of the test dataset
      y_pred = model.predict(X_test)

      #Evaluate the performance of the model
      accuracy = accuracy_score(y_test, y_pred)
      print('Accuracy: ', accuracy)
```

Accuracy: 1.0

Model Optimisation

```

[15]: from sklearn.datasets import load_iris
      from sklearn.tree import DecisionTreeClassifier
      from sklearn.model_selection import train_test_split, GridSearchCV
      from sklearn.metrics import accuracy_score

[16]: #Load the iris dataset
      iris = load_iris()
      X = iris.data
      y = iris.target

      #Split the dataset into training and testing sets
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25,
      ↪random_state = 42)

      #Create a DT classifier
      model = DecisionTreeClassifier()

      #Define the parameter grid search
      param_grid = {
          'criterion': ['gini', 'entropy'],
          'max_depth': [None, 5, 10, 15],
          'min_samples_split': [2, 5, 10],
          'min_samples_leaf': [1, 2, 4]
      }

      #Perform grid search [Cross Validation]
      grid_search = GridSearchCV(model, param_grid, cv = 5)
      grid_search.fit(X_train, y_train)

      #Get the best parameters found from Grid search
      best_params = grid_search.best_params_
      print('Best Parameters: ', best_params)

      #Use the best parameters to train my model
      best_model = DecisionTreeClassifier(**best_params) #Run with quotes, without
      ↪quotes and withh stars and compare the output
      best_model.fit(X_train, y_train)

      #Predict the labels of the test dataset
      y_pred = best_model.predict(X_test)

      #Evaluate the performance of the model
      accuracy = accuracy_score(y_test, y_pred)
      print('Accuracy: ', accuracy)

```

```

Best Parameters: {'criterion': 'entropy', 'max_depth': None,
'min_samples_leaf': 4, 'min_samples_split': 2}
Accuracy: 1.0

```

[]: